

tutorialverse: Interoperability and quality infrastructure for R tutorials

Shaurita D. Hutchins 
University of Alabama at Birmingham

Samuel Bharti 
University of Alabama at Birmingham

Executive summary

R educators can choose among several tools for interactive teaching, but finding a suitable tutorial and checking that it still works remains difficult. Lessons use different formats and describe their requirements inconsistently. An instructor comparing them often has to inspect each repository, check its dependencies, and determine whether anyone still maintains it.

tutorialverse will help instructors find R tutorials, understand their requirements, and see when they were last checked and reviewed. We will build an R package with adapters that connect existing tutorial engines to a common checking interface. Tutorial descriptions and validation reports will use a shared format. A searchable registry will index tutorials in their original repositories, with a community process for reviewing submissions and handling outdated entries.

The 12-month project will initially support learnr, learnr2, and Quarto Live, and index Carpentries-style materials with stated limits on automated checks. Four small example tutorials will test the adapters across teaching contexts. Community feedback helped focus this proposal on shared metadata and checks. Browser execution and static hosting will use existing tools.

Signatories and community support

Shaurita D. Hutchins and Samuel Bharti are PhD candidates at the University of Alabama at Birmingham and will co-lead the project.

Hutchins will lead educational design and community development. She will develop the tutorial specification and accessibility guidance, coordinate the example tutorials, and establish the contribution process. Her training is in genetics, genomics, and bioinformatics, and her work combines computational biology with reproducible R/Bioconductor workflows. She brings experience in open-source training and Carpentries instruction. Her [Prioritizing Candidate Genetic Variants Using GTEx, HuBMAP, and IDG](#) module for the CFDE Training Center includes coding activities that run in the browser and knowledge checks.

Bharti will lead software architecture and engine integration. He will develop the adapters and common validation reports, and connect the package to existing engine checks. His teaching work includes [r-for-beginners](#), an adaptation of Software Carpentry lessons with slides. He also develops the R packages [bioclients](#), [biohttp](#), and [biobouncer](#), and publishes package documentation through GitHub Actions and GitHub Pages.

We will jointly design the package API and registry, agree on governance, and develop the example tutorials.

We requested public feedback through [r/Stats](#) and the [learnr webR discussion](#). Community contributor [@coatless](#) directed us to [learnr2](#) and [Quarto Live](#), prompting us to focus on shared metadata and checks.

Their response addressed related tools, rather than reviewing the proposed schema. We will seek specific feedback on the schema and adapter boundaries from engine maintainers.

We will also consult Carpentries and Bioconductor educators and university and community instructors, adding documented feedback and letters of support as received.

Problem

Instructors need to judge a tutorial’s content, prerequisites, difficulty, and duration, but course sites and package documentation describe these inconsistently. A tutorial may remain online after its exercises stop working or its author stops maintaining it.

learnr runs tutorials as Shiny applications (Aden-Buie et al. 2025). learnr2 builds on Quarto Live, which uses webR to execute R in the browser (learnr2, n.d.; Stagg 2026). tutorialverse will connect these authoring approaches through common tutorial descriptions, validation records, and discovery. Carpentries and Bioconductor materials will provide additional teaching contexts.

Evidence of need and prior work

Educators have long had to compare overlapping tools. Handel (2025) assessed learnr, webexercises, check-down, exams, quarto-webR, and quarto-live against his requirements for hosting, maintenance, and exercise types. Kranz’s 2019 comparison of RTutor and learnr shows that choices about exercise structure, teaching environments, and long-term support predate webR.

Discussions in the learnr community document difficulties integrating custom Shiny questions and a report of Python exercises working locally but failing after deployment. These reports illustrate support needs; they do not establish how often tutorials fail.

The rstats-ed project assembled a user-curated discovery list of R courses and learning materials, acknowledging that it could not be comprehensive. TeSS aggregates training metadata, and Bioschemas provides shared descriptions for training resources (Garcia et al. 2020; Castro et al. 2023). Glittr.org helps users find and compare teaching repositories; its 2024 paper explicitly excludes evaluation of their educational quality (Geest et al. 2024).

The Galaxy Training Network provides a precedent for combining automated checks with community review and ongoing maintenance (Hiltemann et al. 2023).

tutorialverse will build on these approaches, mapping tutorial descriptions to existing standards and recording which exercises and dependencies were checked, in which environment, and when. Its registry will combine these records with review and maintenance information. Existing engines will run the tutorials; the project will not build a new engine, browser runtime, or learning-management system. Static hosting will remain a deployment mode to describe and check.

Proposal

Milestone 1: A minimal metadata schema

We will begin with eight required fields: identifier, title, source URL, lesson URL, engine or format, maintainer, license, and learning objectives. We will map these fields to Schema.org and the Bioschemas TrainingMaterial profile where applicable. Topic, level, prerequisites, packages, deployment mode, duration, accessibility information, and software versions will be optional in the initial schema. Missing values will be explicit; fields needed for a particular check must be supplied or inferred reliably by its adapter before that check can run. Validation dates and results will be generated by the checking tools. Nominations with unknown maintainers or licenses will remain pending review.

We will reuse metadata already present in tutorial sources and use a small accompanying file for missing fields. Authors will not need to rewrite lessons. Engine maintainers and educators will review the draft schema, and the pilot will test the effort needed to describe existing tutorials before version 1.0. We will publish the schema, example records, and adoption guidance.

Milestone 2: Adapters for existing engines

Adapters will translate each engine’s metadata and available checks into the common format. Proposed interfaces include `learnr_adapter()`, `learnr2_adapter()`, and `quarto_adapter()`, with Quarto Live support

in the Quarto adapter. We will refine the interfaces with maintainers and document the versions and features each adapter supports.

An adapter may invoke an engine’s existing build or checking tools, but will not implement its execution system or promise conversion between engines. A capability table will distinguish metadata import, rendering checks, code execution, and accessibility checks. Carpentries-style materials can enter the registry through common metadata and link checks; deeper checks will depend on their build system. This lets the registry include useful resources without implying identical test coverage.

Milestone 3: Validation across engines

`check_tutorial()` will combine common checks with the selected adapter. It will check metadata, links, and required files, then assess declared dependencies and package versions. Where the engine supports it, the adapter will test rendering and executable code and identify missing or duplicate exercise identifiers. Basic automated accessibility checks will accompany human review of keyboard use and instructional clarity.

Reports will identify the failing field or exercise and record the tested environment, source version, and date. Reports will distinguish unsupported or unrun checks from passes and failures across all engines. A reachable page, a successful render, or precomputed output will not count as testing interactive exercises. Passing automated checks will not establish teaching quality or full accessibility.

We will publish a reusable GitHub Actions workflow and integration guidance so authors can run checks in their own repositories without joining the registry. We will test the package on Linux, macOS, and Windows, using examples with known faults to verify that the checks detect problems.

Milestone 4: A registry of independently maintained tutorials

The registry will index tutorial descriptions and link to the original repositories and published lessons. Authors will retain responsibility for their content and hosting. Each registry entry will be a YAML file, with automated ingestion into a searchable website and a published `index.json`. A proposed `tutorial_search()` function will return results as an R data frame.

Readers will filter by topic, package, level, learning objective, engine, and deployment mode where those fields are available. Duration and maintenance status will provide further context where available. Entries will distinguish metadata validity, execution checks, and human review so readers can judge the evidence available for a tutorial. Scheduled checks will flag broken links and outdated validation records.

The initial pilot will index 5–10 existing tutorials across the supported formats. We will record missing fields and manual corrections to assess the effort required from authors. Acceptance will require valid records and working search results, while execution claims will depend on separate adapter tests.

Milestone 5: Review, governance, and example tutorials

Authors will submit entries through structured pull requests, and others will be able to nominate tutorials. Automated checks will precede human review of instructional clarity, accessibility, scope, licensing, and maintenance expectations. Review criteria will draw on open lesson-review practices ([The Carpentries 2022](#)). Published guidance will distinguish nominations from author submissions and explain corrections, removal, and archival decisions. Governance documentation will name responsibilities for maintaining the schema, package, and registry.

We will use four small example tutorials to test the adapters: `renv`, package development with `usethis` and `devtools`, an introductory `DESeq2` workflow, and a Carpentries R companion. We will choose representative exercises and adapt existing material under compatible licenses. Bharti’s `r-for-beginners` lessons and Bioconductor training resources offer starting points.

These tutorials will test metadata mapping, validation reports, and engine-specific checks as development proceeds. Together they will cover the three initial adapters; we will not require every example to run under every engine. Tests will include browser interaction where supported and local or server execution where needed. Three or four small integration templates will show authors how to connect their chosen engine to tutorialverse using their existing lessons.

Project plan

Preliminary work

We have reviewed tutorial engines, registries, and metadata schemas and used [community feedback](#) to focus the project on common metadata and checks. Our prior teaching work includes [Hutchins’s candidate genetic variants module](#) and [Bharti’s r-for-beginners lessons](#).

Schedule and working practices

Period	Work and completion criteria
Months 1–2	Review the eight-field schema, map metadata from existing tutorials into pilot records, establish repositories and governance, and begin the example tutorials.
Months 3–5	Implement the three adapters and <code>check_tutorial()</code> . Publish a capability table and common report format. Test valid examples and known failures.
Months 6–8	Build registry ingestion and search. Publish <code>index.json</code> and the R search interface. Run the pilot records through the package and document author effort.
Months 9–10	Pilot nominations and human review with external contributors. Test stale-entry handling and gather usability feedback.
Months 11–12	Complete example tutorial and integration-template checks. Address high-priority feedback and release version 1.0 of the package, specification, and registry with maintenance guidance.

We will work openly on GitHub, using issues and milestones to track progress and pull requests for code review. We will publish monthly updates and milestone reports and invite community feedback. Development will use `testthat` for tests, `pkgdown` for documentation, and `renv` to reproduce software environments, with R-universe distribution where appropriate.

Code will use the MIT license. Newly authored content and documentation will use CC BY 4.0 where compatible with source obligations; adaptations will retain required attribution and licensing. We will share release announcements and integration guidance with R education communities.

Budget and funding

Requested budget: USD 10,000. This funds 200 hours at USD 50/hour, allocated across the five milestones below. No hardware, cloud, travel, or fee costs are included.

Milestone	Hutchins (h)	Bharti (h)	Total (h)	USD
Minimal schema	20	10	30	1,500
Adapters	5	45	50	2,500
Validation package	15	40	55	2,750
Registry	15	20	35	1,750
Review, governance, and examples	25	5	30	1,500
Total	80	120	200	10,000

If funding is reduced, the schema and validation tools can be used independently of the registry.

Risks and maintenance

Adoption is the main uncertainty. A metadata schema will have little value if authors find it burdensome or it duplicates fields they already maintain. Early consultation and the indexing pilot will test this assumption. We will reuse existing metadata and revise requirements in response to documented author effort before freezing version 1.0.

Engine changes could break adapters or expose differences in test coverage. We will document supported versions and browser or server requirements, and use the example tutorials to detect regressions.

The registry will need ongoing attention. Entries will name maintainers and retain source links, validation dates, and status. Scheduled checks and an archival policy will help contributors address stale resources.

Public contribution and governance guidance will allow others to maintain the project without relying on the two leads for every change.

Success

Authors will be able to describe and check existing tutorials without changing tools. Instructors will be able to compare requirements, validation records, and review status across engines.

The baseline records results currently documented for tutorialverse. We will update it at grant start to include any completed pilot work.

Measure	Documented baseline	Grant-end target
Adapters with capability reports	0 demonstrated	3 initial adapters
Tutorials indexed through tutorialverse	0 demonstrated	5–10 across supported formats
External reviewers of schema, adapters, or tutorials	0 documented reviews	At least 3
Example tutorials tested in CI	0 demonstrated	4, collectively covering all adapters
Integration templates	0 demonstrated	3–4
Reusable tutorial-validation workflow	0 demonstrated	At least 1

Completion also requires version 1.0 of the schema, R package, and registry, with review, governance, and stale-resource policies published and exercised in the pilot.

Validation success will mean passing the checks reported for that engine and environment. During the grant, we will track missing metadata and the work authors need to supply it, alongside external submissions and review participation. After release, validation freshness and broken links will guide maintenance; the packages represented in the registry will help identify gaps in coverage.

Future work could add adapters for other tutorial engines or connect the registry to pkgdown, R-universe, and educational catalogs. Multilingual metadata, richer accessibility checks, and instructor collections remain outside the initial funded scope.

References

- Aden-Buie, Garrick, Barret Schloerke, J. J. Allaire, and Alexander Rossell Hayes. 2025. *learnr: Interactive Tutorials for R*. Released. <https://CRAN.R-project.org/package=learnr>.
- Castro, Leyla Jael, Patricia M. Palagi, Niall Beard, Teresa K. Attwood, and Michelle D. Brazas. 2023. “Bioschemas Training Profiles: A Set of Specifications for Standardizing Training Information to Facilitate the Discovery of Training Programs and Resources.” *PLOS Computational Biology* 19 (6): e1011120. <https://doi.org/10.1371/journal.pcbi.1011120>.
- Garcia, Leyla, Bérénice Batut, Melissa L. Burke, et al. 2020. “Ten Simple Rules for Making Training Materials FAIR.” *PLOS Computational Biology* 16 (5): e1007854. <https://doi.org/10.1371/journal.pcbi.1007854>.
- Geest, Geert van, Yann Haeffliger, Monique Zahn-Zabal, and Patricia M. Palagi. 2024. “Using Glittr.org to Find, Compare and Re-Use Online Materials for Training and Education.” *PLOS ONE* 19 (12): e0308729. <https://doi.org/10.1371/journal.pone.0308729>.
- Handel, Andreas. 2025. *Quizzes and Exercises for Quarto Websites*. <https://www.andreashandel.com/posts/2025-02-25-exercises-in-quarto/>.
- Hiltemann, Saskia, Helena Rasche, Simon Gladman, et al. 2023. “Galaxy Training: A Powerful Framework for Teaching!” *PLOS Computational Biology* 19 (1): e1010752. <https://doi.org/10.1371/journal.pcbi.1010752>.
- learnr2: Interactive R Tutorials*. n.d. <https://github.com/PPBDS/learnr2>.
- Stagg, George. 2026. *quarto-live: WebAssembly Powered Code Blocks and Exercises for R and Python in Quarto Documents*. Released. <https://github.com/r-wasm/quarto-live>.
- The Carpentries. 2022. *Piloting Open Peer Review of Lessons*. <https://carpentries.org/blog/2022/02/carpentries-lab-lesson-review-pilot/>.